TOOLKIT · BACKEND ENGINEERING HANDBOOK

# REDIS COMPLETE HANDBOOK

In-Memory Data Structures, Caching Patterns, Persistence, Eviction, Lua Scripting & Production Reliability

| | |
|---|---|
| **Phase** | II — Backend Infrastructure |
| **Tool ID** | 07 of 13 |
| **Curriculum Scope** | Complete 3-Chunk Comprehensive Curriculum |
| **Deliverable Type** | Consolidated Tool Engineering Handbook (Release Build) |
| **Source Authority** | content/redis/ (01, 02, and 03 markdown sources) |
| **Target Audience** | Backend Software Engineers, SDE-1 / SDE-2, Systems Architects |
| **Engineering Focus** | In-Memory Caching, Dual-Write Consistency, RDB/AOF Persistence, Eviction Policies & Rate Limiting |

This comprehensive handbook provides an authoritative guide to **Redis** for backend engineering. Covering three rigorous learning layers—Mental Models, Practical Engineering, and Production Incident Mastery—it equips developers to design low-latency caching layers with Spring Boot, master core data structures (Strings, Hashes, Lists, Sets, Sorted Sets, Streams), configure RDB and AOF persistence, navigate memory constraints and eviction algorithms, solve the dual-write cache consistency problem, build atomic rate limiters with Lua, diagnose hot keys and slow logs, and handle production outages and thundering herds with resilience.

**Curriculum Structure & Core Topics**

**Part I — Caching Fundamentals, Data Structures & Core Mental Models:** Redis vs PostgreSQL comparison, memory latency, key naming conventions, 6 core data structures, TTL mechanics, cache hits/misses, Docker setup, Spring Data Redis integration, and cache stampede prevention.

**Part II — Persistence, Eviction, TTL Strategy, Caching Patterns & Consistency:** RDB snapshots vs AOF logs, replication vs backups, maxmemory limits, 8 eviction policies (LRU/LFU/Random/TTL), Cache-Aside / Read-Through / Write-Through / Write-Behind, dual-write races, and TTL jitter.

**Part III — Production Reliability, Security, Performance & Incident Mastery:** Atomicity vs pipelining vs transactions (MULTI/EXEC) vs Lua scripts, distributed rate limiting with Lua, hot key diagnosis, Sentinel vs Cluster, connection pooling, security ACLs, and production incident runbooks.

**PART I: CACHING FUNDAMENTALS, DATA STRUCTURES & CORE MENTAL MODELS**

Chunk 1 of 3 · Backend Engineering Handbook

# Redis — Chunk 1: Caching Fundamentals, Data Structures & Core Mental Models

## Backend Engineering Tools & Infrastructure Handbook

**Phase:** II — Backend Infrastructure

**Chunk:** 1 of 3

**Source file:** `content/redis/01-redis-caching-fundamentals-data-structures.md`

---

## Learning Objectives

By the end of this chapter, you should be able to:

- Explain why Redis exists and when a backend should use it.
- Distinguish a cache from a primary database.
- Explain Redis's key-value model and its core data structures.
- Design cache keys and use TTLs correctly.
- Execute essential Redis commands.
- Integrate Redis into a Spring Boot backend.
- Recognize cache misses, stale data, cache stampedes, and common design mistakes.
- Defend your caching decisions in an SDE-1 interview.

---

# Layer 1 — Fundamentals & Mental Models

## 1. What Is Redis?

Redis is an in-memory data structure server that allows applications to store and retrieve data using keys.

Think of it as a very fast shared workspace for data that your application needs frequently.

Suppose you're building an e-commerce backend. A user requests product `P-1042`.

Without caching:

```
Client
  |
  v
Spring Boot API
  |
  v
PostgreSQL
  |
  v
Return product
```

Every request that needs the product queries PostgreSQL.

With Redis caching:

```
Client
  |
  v
Spring Boot API
  |
  v
Redis: Is product:P-1042 cached?
  |
  +---- HIT ----> Return cached product
  |
  +---- MISS ---> PostgreSQL
                    |
                    v
                Store in Redis
                    |
                    v
                Return product
```

Redis can reduce repeated database work and improve response latency when the workload has reusable data.

### Why not query PostgreSQL every time?

PostgreSQL is excellent at durable storage, transactions, relational queries, and consistency. However, repeated queries for the same popular data still consume database resources.

Consider a product page receiving 10,000 requests in a short period. If the product changes infrequently, querying the database for every request may be unnecessary.

A cache allows the application to reuse previously retrieved data.

**Core mental model:** PostgreSQL can remain the source of truth while Redis serves as a fast, temporary copy of selected data.

Redis can also be used for other purposes, including counters, rate limiting, queues, coordination, and ephemeral state. Those use cases require their own design considerations.

## 2. Cache vs. Database

| Property | Redis cache | PostgreSQL |
|---|---|---|
| Primary strength | Fast access to reusable data | Durable relational storage |
| Typical access | Key-based lookups and data structures | SQL queries and transactions |
| Data lifetime | Often limited by TTL or eviction | Usually persists until changed or deleted |
| Data model | Strings, hashes, lists, sets, sorted sets, streams, and more | Tables, rows, relationships |
| Source of truth | Usually not for ordinary cache entries | Often the application's source of truth |
| Failure consequence | Cache loss may increase database load | Data loss can compromise core business records |

These are common patterns, not absolute rules. Redis supports persistence, and some architectures deliberately use it as a primary data store.

For this handbook, our default architecture is:

- **PostgreSQL:** authoritative business data.
- **Redis:** cached results and selected ephemeral state.
- **Spring Boot:** application logic that coordinates both.

Never assume that data stored in Redis is automatically safe to lose or safe to reconstruct. That depends on the use case.

## 3. What Makes Redis Fast?

Redis commonly keeps active data in memory and processes commands through an efficient server architecture.

Three ideas matter:

1. **Memory access:** retrieving data from memory usually avoids the latency of a database disk-backed query path.
2. **Purpose-built data structures:** Redis provides operations for strings, sets, sorted sets, hashes, and other structures rather than requiring every operation to be implemented in application code.
3. **Low-overhead access:** a simple key lookup can be much cheaper than repeatedly executing an expensive database query.

But don't reduce Redis to “everything is O(1).” Complexity depends on the command and the data structure. For example, retrieving one hash field differs from retrieving every member of a large collection.

Redis is also not infinitely fast. Network latency, serialization, large values, memory pressure, slow commands, contention, and excessive request volume can all become bottlenecks.

## 4. The Redis Key-Value Model

Redis stores values under keys.

Conceptually:

```
Key                         Value
-----------------------------------------------
product:1042                {"name":"Keyboard",...}
user:87:profile             {"name":"Aafaq",...}
session:abc123              session data
inventory:1042              42
```

A key should communicate the resource and purpose of its value.

Prefer:

```
product:1042
user:87:profile
order:9001:summary
```

Avoid vague keys:

```
data
user
result
temp
```

Good keys improve debugging, ownership, observability, and maintenance.

### Key naming conventions

A common convention is:

```
<domain>:<identifier>:<purpose>
```

Examples:

```
product:1042:details
product:1042:price
user:87:permissions
catalog:category:electronics
```

Choose one consistent convention across your application.

Do not put secrets, access tokens, passwords, or unnecessary personal information into key names. Keys may appear in diagnostics, monitoring, or operational tooling.

## 5. Redis Data Structures

Redis is more than a string-to-string map. Its data structures help solve different backend problems.

### 5.1 Strings

Strings store a value associated with a key. Values can represent text, serialized JSON, or numbers.

```
SET product:1042:name "Mechanical Keyboard"
GET product:1042:name
```

Example response:

```
"Mechanical Keyboard"
```

A numeric counter:

```
SET page:home:views 0
INCR page:home:views
INCR page:home:views
GET page:home:views
```

The final value is **2**.

Redis provides atomic increment operations. This is useful when multiple requests need to update the same counter without implementing a read-modify-write sequence in application code.

**Backend use cases:**

- Cached JSON responses.
- Simple counters.
- Feature flags.
- Short-lived verification state.
- Rate-limit counters.

## 5.2 Hashes

Hashes store fields and values under a single key.

```
HSET user:87:profile name "Aafaq" role "developer"
HGET user:87:profile name
HGETALL user:87:profile
```

Conceptually:

```
user:87:profile
  |
  +-- name = "Aafaq"
  +-- role = "developer"
```

Hashes are useful when a resource naturally consists of several fields.

However, don't assume a Redis hash is always superior to a serialized JSON string. Choose based on how the application accesses and updates the data, and consider schema evolution, serialization, and the number of network calls.

## 5.3 Lists

Lists hold ordered sequences of values.

```
LPUSH notifications:user:87 "Welcome"
LPUSH notifications:user:87 "Order shipped"

LRANGE notifications:user:87 0 -1
```

Lists support operations at their ends and can be used for simple ordered collections or queue-like patterns.

Possible use cases:

- Recent activity.
- Simple work queues.
- Ordered messages.

For robust distributed event processing, Redis Lists are not automatically a replacement for a purpose-built system such as Kafka. Delivery guarantees, consumer coordination, replay, retention, and operational requirements must be considered.

### 5.4 Sets

Sets hold unique members.

```
SADD user:87:roles "developer" "reviewer"
SADD user:87:roles "developer"

SMEMBERS user:87:roles
SISMEMBER user:87:roles "reviewer"
```

Adding `developer` twice does not create two distinct members.

Useful for:

- Unique membership.
- Tags.
- Feature eligibility.
- Set intersection and union operations.

For example, finding users who belong to both groups:

```
SINTER team:backend team:oncall
```

### 5.5 Sorted Sets

Sorted sets associate unique members with scores.

```
ZADD leaderboard 1200 "player:1"
ZADD leaderboard 950 "player:2"
ZADD leaderboard 1500 "player:3"

ZREVRANGE leaderboard 0 2 WITHSCORES
```

The reverse range returns the highest-scoring members first.

Useful for:

- Leaderboards.
- Rankings.
- Priority ordering.
- Time-based indexes when designed appropriately.

Sorted sets are especially helpful when the application needs ordering by a numeric score without fetching and sorting the entire dataset itself.

### 5.6 Streams

Redis Streams provide an append-oriented data structure for event records and consumer-group workflows.

They support use cases such as:

- Event ingestion.
- Consumer groups.
- Reading events from an ID.
- Tracking and acknowledging processed entries.

Streams are more capable than a simple list for event-processing workflows, but they still require careful choices about retention, consumer recovery, duplicate handling, and delivery semantics.

### Quick data-structure selection

| Requirement | Candidate |
|---|---|
| Cache one object or response | String containing serialized data |
| Read or update individual fields | Hash |
| Maintain an ordered sequence | List |
| Store unique members | Set |
| Rank members by score | Sorted set |
| Append and consume event records | Stream |
| Maintain a numeric counter | String with `INCR`/`INCRBY` |

These are starting points, not rigid rules. Select the structure that best matches the required operations.

## 6. TTL: Time to Live

A TTL determines how long a key should remain before it expires automatically.

```
SET product:1042:details '{"name":"Keyboard"}' EX 300
```

The key is configured to expire after 300 seconds.

Inspect the remaining TTL:

```
TTL product:1042:details
```

Common outcomes:

- A positive number: remaining lifetime in seconds.
- `-1`: the key exists but has no expiration.
- `-2`: the key does not exist.

You can set an expiry separately:

```
EXPIRE product:1042:details 300
```

Or inspect the stored value:

```
GET product:1042:details
```

### Why TTL matters

Imagine that a product's price changes in PostgreSQL, but Redis keeps the old cached value forever. Customers could continue seeing stale pricing.

A TTL bounds the lifetime of that cached copy, assuming it is not refreshed or replaced.

However, TTL is **not an instantaneous consistency guarantee**. A value may remain stale until it expires, and expiry alone does not solve every race between concurrent reads and writes.

A shorter TTL generally reduces the maximum cache lifetime but increases database traffic. A longer TTL may improve cache hit rates but can prolong staleness.

TTL is a design decision, not an arbitrary number.

## 7. Cache Hits and Cache Misses

A **cache hit** occurs when the requested value is present in Redis.

A **cache miss** occurs when the value is absent, expired, or otherwise unavailable for the requested lookup.

Suppose an endpoint retrieves a product:

```
Request 1:
Redis lookup -> MISS
PostgreSQL query -> product found
Store result in Redis
Return product

Request 2:
Redis lookup -> HIT
Return cached product
```

The first request performs the database lookup. The second can reuse the cached value.

### Cache hit ratio

A simple cache hit ratio is:

\[

\text{Hit Ratio} =

\frac{\text{Cache Hits}}

{\text{Cache Hits}+\text{Cache Misses}}

\times 100\%

\]

If 800 lookups hit the cache and 200 miss:

\[

\frac{800}{800+200}\times100=80\%

\]

An 80% hit ratio does not automatically mean the system is healthy. A cache could have a high hit ratio while returning stale data, consuming excessive memory, or causing high latency for misses.

Evaluate hit ratio alongside latency, error rates, database load, and data correctness.

---

# Layer 2 — Practical Engineering & Workflows

## 8. Start Redis Locally with Docker

If Docker is available, start a disposable Redis instance:

```
docker run --name redis-handbook \
  -p 127.0.0.1:6379:6379 \
  -d redis:7
```

This example pins the major Redis version rather than using an unspecified latest tag. For a real project, choose a supported version and pin an appropriate image tag or digest according to your release policy.

Check that the container is running:

```
docker ps
docker logs redis-handbook
```

Open a Redis CLI session:

```
docker exec -it redis-handbook redis-cli
```

Test connectivity:

```
PING
```

Expected response:

```
PONG
```

Run a few commands:

```
SET greeting "Hello Redis"
GET greeting
EXPIRE greeting 60
TTL greeting
```

Clean up the container when finished:

```
docker rm -f redis-handbook
```

This command removes the container. Do not use this disposable setup for production data.

**Important:** this example intentionally binds Redis to loopback on the host. Do not expose an unauthenticated Redis instance directly to an untrusted network.

## 9. Essential Redis CLI Commands

Keep this command set handy.

| Command | Purpose |
|---|---|
| `PING` | Check basic server responsiveness |
| `SET key value` | Store a string |
| `GET key` | Read a string |
| `DEL key` | Delete a key |
| `EXISTS key` | Check whether a key exists |
| `EXPIRE key seconds` | Set an expiry |
| `TTL key` | Inspect remaining lifetime |
| `INCR key` | Increment a numeric string |
| `HSET key field value` | Set a hash field |
| `HGET key field` | Read a hash field |
| `SADD key member` | Add a set member |
| `SMEMBERS key` | Read set members |
| `LRANGE key start stop` | Read a list range |
| `ZADD key score member` | Add a sorted-set member |
| `ZREVRANGE key start stop WITHSCORES` | Read highest-scoring members first |
| `SCAN 0` | Begin incremental key iteration |

### Why `SCAN` instead of `KEYS *`?

`KEYS *` can traverse the entire keyspace in one command and can cause disruptive latency on a busy server.

`SCAN` supports incremental iteration, allowing clients to process batches over multiple calls. It is not a perfectly consistent snapshot, and clients must handle repeated results and cursor progression correctly.

In application code, prefer purpose-built key naming, known keys, and indexed structures over routinely searching the entire keyspace.

## 10. Design a Cache-Aside Workflow

Cache-aside is a common caching pattern:

1. Application checks Redis.
2. On a hit, return the cached value.
3. On a miss, read the source database.

4. Populate Redis with the result.
5. Return the result to the caller.

The application explicitly manages cache reads and population.

### Example: product lookup

Assume PostgreSQL is the source of truth for products.

```
public Product getProduct(long productId) {
    String key = "product:" + productId;

    Product cached = redisCache.get(key, Product.class);

    if (cached != null) {
        return cached;
    }

    Product product = productRepository.findById(productId)
        .orElseThrow(() -> new ProductNotFoundException(productId));

    redisCache.put(key, product, Duration.ofMinutes(5));

    return product;
}
```

This example illustrates the flow; `redisCache` is an application abstraction that you would implement using Spring Data Redis or another Redis client.

The critical behavior is the ordering:

- Check Redis first.
- Query PostgreSQL on a miss.
- Populate Redis after a successful database read.

The exact treatment of missing records, serialization failures, and Redis outages depends on the endpoint's requirements.

### Why not cache every database query?

Some data is:

- Rarely accessed.
- Frequently modified.
- Expensive to serialize.
- Sensitive.
- Too large to justify retaining.
- Difficult to invalidate safely.

Cache selectively. Start with a known bottleneck or high-read workload and measure whether caching improves it.

## 11. Spring Boot Integration

For a Spring Boot application, use Spring Data Redis and a compatible Redis client.

With Maven, the Spring Boot starter is commonly declared as:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

Let your Spring Boot dependency management select a compatible version rather than hardcoding an arbitrary version.

Configure the connection using environment variables:

```
spring:
  data:
    redis:
      host: ${REDIS_HOST:localhost}
      port: ${REDIS_PORT:6379}
```

The exact configuration properties can vary with Spring Boot versions and the selected Redis client. Verify them against the version used by your project.

## A small cache service

A simplified example using **`StringRedisTemplate`**:

```
@Service
public class ProductCacheService {

    private final StringRedisTemplate redis;

    public ProductCacheService(StringRedisTemplate redis) {
        this.redis = redis;
    }

    public void put(String key, String value, Duration ttl) {
        redis.opsForValue().set(key, value, ttl);
    }

    public String get(String key) {
        return redis.opsForValue().get(key);
    }

    public void evict(String key) {
        redis.delete(key);
    }
}
```

This example stores strings. A production application that caches Java objects must choose a serialization format and configure it consistently.

Prefer explicit, understandable serialization. Avoid unsafe polymorphic deserialization configurations, and treat cached data as untrusted input if its source or integrity is not guaranteed.

## Test the integration

1. Start Redis locally.
2. Start the Spring Boot application.
3. Store a value with a short TTL.

4. Read it and confirm the value matches.
5. Wait for expiration and confirm it is absent.
6. Stop Redis and observe the application's behavior.
7. Restart Redis and verify that the application can reconnect according to its configuration.

This last pair of tests is important: successful integration does not mean the system behaves correctly when Redis becomes unavailable.

## 12. Cache Keys and Expiration in a Backend API

Suppose you have these endpoints:

```
GET /api/products/1042
GET /api/products/1043
GET /api/users/87/profile
```

Use distinct keys:

```
product:1042:details
product:1043:details
user:87:profile
```

A product lookup should not accidentally reuse another product's cached value. A user-specific response should not be shared across users just because the URL or key was designed poorly.

For cache keys, consider:

- Resource identity.
- Tenant or organization identity where applicable.
- Locale or representation version when the response differs by locale or schema.
- Relevant query parameters for list endpoints.
- Authorization boundaries.
- Schema changes and cache-key versioning.

For example:

```
v1:tenant:42:product:1042:details
```

A versioned prefix can make a schema transition easier to manage, although it may temporarily leave old entries in Redis until they expire or are removed.

### Cache negative results carefully

If a product does not exist, repeated requests can still generate database load. Some applications cache a “not found” result briefly to reduce repeated lookups.

But negative caching requires care:

- Use a short, deliberate TTL.
- Distinguish a cached negative result from a cache miss.
- Invalidate the negative result when the resource is created.
- Avoid turning temporary database failures into cached “not found” responses.

A database timeout does not mean that a product does not exist.

## 13. Hands-On Lab: Build a Mini Product Cache

Complete the lab in Redis CLI.

### Task A — Cache a product

```
SET product:1042:details '{"id":1042,"name":"Keyboard","price":2499}' EX 120
GET product:1042:details
TTL product:1042:details
```

Verify that the value exists and has a positive TTL.

### Task B — Update the cached value

```
SET product:1042:details '{"id":1042,"name":"Keyboard","price":2299}' EX 120
GET product:1042:details
```

Observe that `SET` replaces the previous value and, with `EX 120`, establishes a new expiry.

### Task C — Build a counter

```
SET product:1042:view-count 0
INCR product:1042:view-count
INCR product:1042:view-count
GET product:1042:view-count
```

Expected final value:

```
2
```

### Task D — Inspect expiration

```
SET demo:short-lived "temporary" EX 5
TTL demo:short-lived
```

Wait several seconds, then run:

```
GET demo:short-lived
```

Expected result after expiration: a nil response.

### Task E — Think like an engineer

Answer these questions:

1. Which component owns the authoritative product price?

**2.** What happens if PostgreSQL updates the price but Redis retains the previous value?

**3.** How would you invalidate the cached product after an update?

**4.** What should the API do if Redis is unavailable?

**5.** What happens when 1,000 concurrent requests miss the same popular key?

The final question leads directly to the production concerns explored in later chapters.

---

# Layer 3 — Production Engineering & Interview Mastery

## 14. The Cache-Aside Pattern Is Not a Consistency Guarantee

Cache-aside is easy to understand, but concurrency creates edge cases.

Imagine:

```
Request A                 Request B
---------                 ---------
Read old product from DB
                          Update product in DB
                          Delete Redis key
Write old product to Redis
```

The final Redis entry can contain stale data even though Request B invalidated the key.

This is a race condition. Simple “update database, then delete cache” logic does not eliminate every possible stale-write scenario.

For many read-heavy endpoints, a short TTL and cache invalidation may be an acceptable trade-off. For stricter requirements, the design may need versioned entries, coordinated updates, or a different consistency strategy.

Do not promise strong consistency simply because your application deletes a key after a database update.

## 15. Common Production Failure Modes

### Failure 1 — Cache stampede

A popular key expires, and many requests simultaneously query the database to refill it.

```
1,000 requests
      |
      v
Same cache miss
      |
      v
1,000 database queries
```

Potential mitigations include:

- Request coalescing or single-flight loading.
- A short randomized TTL variation to avoid synchronized expirations.

- Background refresh for suitable data.
- Carefully designed locking when its overhead and failure modes are understood.
- Database capacity controls.

The right solution depends on the workload. A distributed lock is not automatically the best first choice.

### Failure 2 — Stale cached data

The source database changes, but the cache is not updated or invalidated correctly.

Mitigations:

- Define invalidation behavior for every relevant write path.
- Choose TTLs based on freshness requirements.
- Test concurrent reads and writes.
- Monitor stale-data symptoms where measurable.

### Failure 3 — Redis outage

The application cannot read or write cached values.

A common strategy is to fall back to PostgreSQL for cache reads, provided the database has enough capacity. Cache writes may be skipped temporarily when they are not required for correctness.

But consider the consequences: if all requests suddenly hit PostgreSQL, a Redis outage can overload the database.

Mitigations:

- Bound Redis connection and command timeouts.
- Use connection pooling appropriately.
- Apply database-side concurrency limits and application load shedding where necessary.
- Monitor database saturation during cache degradation.
- Decide whether each endpoint should fail open or fail closed.

“Fail open” means continuing without the cache where safe. It does not mean bypassing authentication, authorization, or other essential controls.

### Failure 4 — Memory pressure

Redis memory is finite. If the workload exceeds available memory, performance or write behavior depends on configuration and the selected eviction policy.

Possible causes:

- Unbounded keys.
- Missing TTLs.
- Oversized cached values.
- Excessive duplication.
- A workload whose working set no longer fits.

Later Redis chapters will cover eviction policies, memory accounting, persistence, and deeper operational diagnostics.

### Failure 5 — Cache key collision or tenant leakage

A key does not include enough context to distinguish two resources or authorization boundaries.

For example, caching a response under:

```
profile:current
```

is dangerous if that key is shared across different users.

Prefer identity-aware keys and ensure cached responses respect authorization. A cache hit must never bypass the checks required to decide whether a caller may access a resource.

### Failure 6 — Cache miss treated as a database failure

A missing cache entry is normally an expected condition, not an application error.

Correct logic:

```
Redis key absent
      |
      v
Read from PostgreSQL
```

Incorrect logic:

```
Redis key absent
      |
      v
Return HTTP 500
```

A cache miss should trigger the normal loading path unless the application's design explicitly defines otherwise.

## 16. Choosing a Cache TTL

There is no universally correct TTL.

| Data | Possible starting approach | Important consideration |
|---|---|---|
| Product details | Minutes | How quickly must catalog changes appear? |
| Public configuration | Minutes or longer | How are configuration updates propagated? |
| User profile | Short TTL or explicit invalidation | How sensitive is profile freshness? |
| Real-time inventory | Very short TTL or no cache for critical reads | Can stale stock cause overselling? |
| Search results | Short TTL | Query diversity and invalidation complexity |
| Authentication or authorization state | Carefully designed, if cached at all | Revocation and security requirements |

These are illustrative strategies, not universal production defaults.

For business-critical values such as inventory or prices, decide which operations require authoritative reads or stronger consistency. Do not rely on TTL alone when stale data could cause unacceptable business outcomes.

## 17. Measure Whether Caching Helps

A cache should improve a measurable outcome.

Track relevant metrics such as:

- Cache hit and miss counts.
- Hit ratio.
- Redis command latency.
- Redis errors and connection failures.
- Redis memory usage.
- Key expirations and evictions.
- PostgreSQL query latency and connection utilization.
- API p50, p95, and p99 latency.
- Request volume during cache degradation.

For example, compare the same endpoint before and after introducing Redis:

| Measurement | Before cache | After cache |
|---|---|---|
| API p95 latency | Measure | Measure |
| Database queries per second | Measure | Measure |
| Cache hit ratio | N/A | Measure |
| Error rate | Measure | Measure |

Do not invent benchmark improvements. Run a controlled test with a representative workload, comparable data, and clearly defined measurement conditions.

A high hit ratio is useful only if it translates into meaningful improvements without unacceptable correctness or operational costs.

## 18. Security Basics

Treat Redis as an internal infrastructure service, not a public API.

Baseline precautions:

- Keep Redis on a private network.
- Require authentication and use ACLs where appropriate.
- Use TLS when required by your network and security architecture.
- Restrict network access to authorized application services.
- Avoid placing credentials in source code or committed configuration.
- Protect sensitive cached values and minimize unnecessary retention.
- Avoid untrusted serialized object deserialization.
- Use separate credentials or access policies when applications have different trust boundaries.
- Keep Redis and its client libraries updated.

Redis's availability on an internal network does not make every command safe for every caller. Apply least privilege and limit operational access.

## 19. SDE-1 Interview Questions & Answers

### Q1. What is Redis?

Redis is an in-memory data structure server commonly used for caching, counters, ephemeral state, coordination, and selected data-processing workloads.

### Q2. Why is Redis commonly used as a cache?

It can serve frequently accessed data quickly and reduce repeated work on the primary database.

### Q3. What is a cache hit?

The requested value is present in the cache and can be served from it.

### Q4. What is a cache miss?

The requested value is absent or unavailable, so the application generally follows a fallback path such as querying the database.

### Q5. What is TTL?

Time to live is the configured lifetime of a key before it expires.

### Q6. What is cache-aside?

The application checks the cache first, loads missing data from the source database, and populates the cache.

### Q7. Redis vs. PostgreSQL?

Redis provides fast access to in-memory data structures and is often used as a cache. PostgreSQL provides relational storage, SQL queries, and transactional guarantees. They solve different problems.

### Q8. When would you use a Redis hash?

When a resource is naturally represented as fields under one key and those fields need to be accessed or updated individually.

### Q9. When would you use a sorted set?

When members need to be ranked or ordered by scores, such as in a leaderboard.

### Q10. What is a cache stampede?

Many requests attempt to reload the same missing or expired key concurrently, potentially overwhelming the backing database.

### Q11. Does a high cache hit ratio guarantee good performance?

No. Latency, error rates, memory pressure, stale data, and database load also matter.

### Q12. What happens if Redis goes down?

Cache reads and writes may fail. The application needs a deliberate degradation strategy that accounts for database capacity and endpoint correctness.

### Q13. Why should cache keys follow a naming convention?

Consistent names improve isolation, troubleshooting, observability, and maintainability.

### Q14. Why can `KEYS *` be dangerous?

It may traverse a large keyspace in one operation and introduce disruptive latency. Incremental iteration with `SCAN` is generally more suitable for operational inspection.

### Q15. Is Redis always the source of truth?

No. In many backend systems, a relational database remains authoritative while Redis stores a rebuildable copy. Redis can serve as primary storage in other architectures, depending on the durability and consistency requirements.

---

## 20. Mastery Assessment

Try these before moving to Chunk 2.

**1. Design:** Your product endpoint receives 5,000 reads per minute for the same product. Would you consider Redis? What would you measure before and after?

**2. Debugging:** Redis returns a value, but the value is older than PostgreSQL. Name two plausible causes and explain how you would investigate.

**3. Reliability:** Redis becomes unavailable during peak traffic. What happens to PostgreSQL load if every API request falls back to the database?

**4. Data structures:** Choose an appropriate Redis structure for each:

- A page-view counter.
- A leaderboard.
- Unique members of a group.
- A user's individual profile fields.
- An ordered event-consumption workflow.

**5. Design trade-off:** Why might a two-minute TTL be reasonable for product descriptions but inappropriate for certain inventory checks?

**6. Code review:** Identify the risk in a cache key that omits a tenant ID when multiple tenants can have resources with the same local identifier.

### Completion checklist

- [ ] I can explain Redis without describing it merely as a database replacement.

- [ ] I can distinguish a cache hit from a cache miss.
- [ ] I can explain cache-aside and TTL.
- [ ] I can choose between strings, hashes, lists, sets, sorted sets, and streams.
- [ ] I can execute the core Redis CLI commands.
- [ ] I can integrate a basic Redis cache into Spring Boot.
- [ ] I understand why stale data and cache stampedes occur.
- [ ] I can explain how a Redis outage can increase database load.
- [ ] I can identify key-naming and authorization risks.
- [ ] I can describe what to measure to prove caching is beneficial.

---

## What Comes Next?

**Redis — Chunk 2 of 3: Persistence, Eviction, TTL Strategy, Caching Patterns & Consistency**

The next chapter moves beyond basic caching into memory management, eviction policies, persistence trade-offs, cache invalidation, write-through versus write-behind approaches, and consistency under concurrent updates.

**Repository workflow:** Save this chapter as `content/redis/01-redis-caching-fundamentals-data-structures.md`. Do not generate a PDF yet. The complete Redis PDF should be generated only after all three Redis chapters have been saved and verified.

**PART II: PERSISTENCE, EVICTION, TTL STRATEGY, CACHING PATTERNS & CONSISTENCY**

Chunk 2 of 3 · Backend Engineering Handbook

# Redis — Chunk 2: Persistence, Eviction, TTL Strategy, Caching Patterns & Consistency

### Backend Engineering Tools & Infrastructure Handbook

**Phase:** II — Backend Infrastructure

**Chunk:** 2 of 3

**Source file:** `content/redis/02-redis-persistence-eviction-caching-patterns.md`

---

### Learning Objectives

By the end of this chapter, you should be able to:

- Explain Redis persistence using RDB snapshots and AOF logs.
- Understand Redis memory limits and eviction policies.
- Design TTL and invalidation strategies for different workloads.
- Compare cache-aside, read-through, write-through, and write-behind.
- Reason about stale data, concurrent updates, and cache consistency.
- Diagnose common cache failures using commands and application workflows.
- Explain Redis caching trade-offs in backend interviews.

---

# Layer 1 — Fundamentals & Mental Models

### 1. Redis Is In-Memory, But Does That Mean Data Disappears on Restart?

Not necessarily.

Redis is designed for fast access to data structures held in memory, but it also supports persistence mechanisms that can write data to disk.

The important distinction is:

- **Memory:** where Redis serves its data during normal operation.

- **Persistence:** how Redis records data so it may be reconstructed after a restart.
- **Replication:** how data is copied to other Redis servers.
- **Backup:** how recoverable copies are retained outside the immediate running instance.

These are different capabilities. Enabling persistence does not automatically provide a complete backup strategy or guarantee zero data loss.

For ordinary caching, data may be reconstructible from PostgreSQL. For Redis workloads that hold important state, persistence and recovery requirements must be designed explicitly.

## 2. RDB Snapshots

RDB persistence stores point-in-time snapshots of the Redis dataset.

Conceptually:

```
Redis dataset in memory
          |
          v
   Snapshot operation
          |
          v
       dump.rdb
```

Redis can be configured to produce snapshots according to specified save conditions, and administrators can trigger snapshots when appropriate.

### Advantages

- Compact snapshot files in many workloads.
- Convenient for restoring a dataset.
- Useful for backup and transfer workflows.

### Trade-offs

A snapshot represents the dataset at a particular point in time. Changes made after the latest successful snapshot may not be present in the recovered dataset.

Snapshot creation also consumes resources. Depending on the operation and workload, it can affect memory use, CPU use, disk I/O, and latency.

**Mental model:** RDB is like taking a photograph of the dataset periodically.

It is useful when recovering to a recent snapshot is acceptable, but the recovery-point objective must be understood.

## 3. AOF — Append-Only File

AOF persistence records write operations so Redis can reconstruct state by replaying the recorded commands.

```
Write command
     |
     v
Redis updates data
     |
     v
AOF records write history
     |
     v
Recovery replays recorded operations
```

The exact recording and durability behavior depends on the AOF configuration and the operating system's handling of disk writes.

### Advantages

- Can reduce the amount of recent data lost compared with infrequent snapshots, depending on the configured fsync policy.
- Records a sequence of write operations.
- Supports recovery by replaying that history.

### Trade-offs

- AOF files may grow and require rewriting.
- Recovery can take time when the log is large.
- Durability depends on configuration and the storage system.
- Disk I/O and rewrite activity can affect resource consumption.

Redis supports different AOF fsync policies, including policies that sync on every write, periodically, or leave syncing to the operating system. They represent different durability/performance trade-offs.

**Mental model:** RDB is a snapshot; AOF is a record of changes.

Some deployments use both. The right choice depends on whether Redis data is disposable cache state or important application state.

## 4. Persistence vs. Replication vs. Backup

These concepts are often confused.

| Capability | Primary purpose | What it does not automatically guarantee |
|---|---|---|
| RDB | Recover from a snapshot | Zero data loss after the snapshot |
| AOF | Reconstruct state from recorded writes | No loss under every failure scenario |
| Replication | Copy data to replicas | A replica is not necessarily an independent backup |
| Backup retention | Preserve recoverable copies | Successful restoration unless tested |

A replica may receive an accidental deletion or corrupted logical state. If that change propagates, the replica does not necessarily preserve the old correct data.

A robust recovery plan defines:

- Recovery Point Objective (RPO): how much data loss is acceptable.

- Recovery Time Objective (RTO): how quickly service must be restored.
- Backup retention and isolation.
- Restoration procedures and testing.
- Monitoring for failed backups and replication problems.

For a disposable product cache, rebuilding from PostgreSQL may be sufficient. For state that cannot be reconstructed, the recovery design must be much more rigorous.

## 5. Redis Memory Is a Finite Resource

Redis performance depends partly on whether the working dataset fits comfortably within the available memory budget.

Suppose the host has 8 GB of RAM. That does not mean the Redis process can safely use all 8 GB.

You must account for:

- Redis dataset memory.
- Per-key and per-value overhead.
- Client connections and buffers.
- Replication buffers.
- Persistence-related memory requirements.
- Operating system and other services.
- A safety margin for workload spikes.

Large numbers of tiny keys can still consume substantial memory because keys and data structures have overhead.

### Inspect memory usage

```
INFO memory
```

Useful fields include:

- **`used_memory`**
- **`used_memory_human`**
- **`used_memory_peak_human`**
- **`maxmemory`**
- **`maxmemory_policy`**

Inspect key-level information:

```
MEMORY USAGE product:1042:details
```

This helps estimate the memory attributable to a particular key, but it is not a complete capacity-planning method.

### Key lesson

Memory planning should be based on representative workloads and measured data, not just the raw size of serialized values.

## 6. What Happens When Redis Runs Out of Memory?

Redis can be configured with a `maxmemory` limit and an eviction policy.

Conceptually:

```
New write arrives
       |
       v
Would memory exceed the configured limit?
       |
       +---- No ----> Process write
       |
       +---- Yes ---> Apply configured policy
                          |
                          +--> Evict eligible keys
                          |
                          +--> Reject write
```

The exact behavior depends on the policy, the command, and whether keys are eligible for eviction.

A cache workload often needs a policy that allows Redis to remove cached entries under memory pressure. Other workloads may need a policy that preserves existing data and rejects writes instead.

## 7. Redis Eviction Policies

Redis supports multiple eviction policies. The following are the major concepts to understand.

| Policy | Behavior |
|---|---|
| `noeviction` | Does not evict keys; writes that need additional memory may fail |
| `allkeys-lru` | Evicts keys selected by an approximate least-recently-used policy across keys |
| `allkeys-lfu` | Evicts keys selected by an approximate least-frequently-used policy across keys |
| `volatile-lru` | Applies approximate LRU eviction to keys with an expiry |
| `volatile-lfu` | Applies approximate LFU eviction to keys with an expiry |
| `allkeys-random` | Evicts randomly selected keys from the keyspace |
| `volatile-random` | Evicts randomly selected keys that have an expiry |
| `volatile-ttl` | Prefers keys with shorter remaining TTLs among eligible keys |

LRU approximates recency: which data has been used recently?

LFU approximates frequency: which data has been accessed frequently?

Neither policy perfectly predicts future access. Their suitability depends on workload patterns.

### Example: a product catalog

Suppose 20% of products account for 80% of requests. An LFU-like strategy may retain frequently accessed products, while less popular entries become candidates for eviction.

But if a campaign suddenly makes a previously unpopular product extremely popular, historical access patterns may not immediately reflect the new workload.

### A subtle but important point

`volatile-*` policies consider only keys that have an expiration set. If the workload has no eligible keys, the system may not be able to evict a key under those policies, and writes requiring memory can fail.

Do not choose an eviction policy without understanding which keys have TTLs and whether eviction is allowed for the workload.

## 8. Cache Expiration Is Different From Eviction

These two concepts are related but distinct.

**Expiration:** a key's configured lifetime has elapsed.

**Eviction:** Redis removes a key to manage memory pressure according to its configured policy.

A key may disappear because its TTL expired even when memory is plentiful. A key may also be evicted before its TTL expires when memory pressure and the configured policy allow it.

This means that a TTL is not a promise that the key will remain present for its entire configured lifetime.

Applications must treat cached entries as optional and handle missing values correctly.

## 9. Cache Invalidation: The Hard Part

Imagine an application updates a product's price in PostgreSQL.

The Redis cache still contains the old price.

There are several common strategies.

### Strategy A — Delete the cache key after a database update

```
Update PostgreSQL
       |
       v
Commit transaction
       |
       v
Delete Redis key
```

A later read misses the cache and reloads the current database value.

This is a common starting point, but it is not perfectly race-free. A concurrent reader could fetch an old value before the update commits and write it into Redis after the invalidation.

### Strategy B — Update the cache after the database update

```
Update PostgreSQL
       |
       v
Commit transaction
       |
       v
Update Redis value
```

This may reduce the time a cache remains stale, but failure between the database commit and the cache update can leave the cache inconsistent.

### Strategy C — TTL-based eventual freshness

Allow the entry to expire and be reloaded later.

This is simple but permits stale values during the TTL window. It is unsuitable when the business requires stronger freshness guarantees.

### Strategy D — Versioned cache entries

Associate cached data with a version or generation identifier.

Versioning can help avoid some stale-write races, but it requires careful handling of version ownership, atomicity, and old-entry cleanup.

No invalidation strategy is universally correct. Pick one that matches the business requirements and document its failure behavior.

## 10. Cache-Aside, Read-Through, Write-Through and Write-Behind

These patterns describe how application reads and writes interact with a cache.

### Cache-aside

The application manages cache lookups and population.

```
Read:
Cache -> miss -> Database -> Populate cache

Write:
Database update -> Invalidate or update cache
```

**Advantages:** simple, flexible, widely used.

**Trade-offs:** application code must handle misses, invalidation, and concurrency.

### Read-through

The application reads through a cache abstraction, and the cache-loading layer retrieves data from the backing store on a miss.

```
Application -> Cache layer -> Database on miss
```

**Advantages:** centralizes loading behavior.

**Trade-offs:** requires a suitable cache abstraction and does not remove the need to reason about invalidation and consistency.

### Write-through

A write is applied to the cache and backing store through a coordinated write path.

```
Application
     |
     v
Coordinated cache/write layer
     |
     +-----> Cache
     |
     +-----> Database
```

The ordering and failure semantics depend on the implementation. A diagram alone does not make two independent stores transactionally atomic.

**Advantages:** can keep cache updates close to write operations.

**Trade-offs:** extra write latency and cross-store failure handling.

### Write-behind (write-back)

The cache accepts a write first, and the backing store is updated asynchronously later.

```
Application
     |
     v
Cache / pending write state
     |
     v
Asynchronous persistence
     |
     v
Database
```

**Advantages:** can reduce synchronous write latency and batch downstream writes.

**Trade-offs:** increased complexity, delayed durability, potential data loss, ordering problems, and recovery requirements.

Do not use write-behind for critical business records merely because it appears faster. It needs a reliable mechanism for tracking, retrying, and recovering pending writes.

## 11. The Database-Cache Dual-Write Problem

A common backend bug occurs when one operation must update two independent systems.

For example:

```
1. Update product in PostgreSQL
2. Update Redis
```

What if step 1 succeeds and step 2 fails?

Or what if Redis is updated first and PostgreSQL fails?

Without a coordinated consistency strategy, the two systems can disagree.

For ordinary caches, deleting the cache after a successful database commit is often a practical baseline. If invalidation fails, the system may need retries, monitoring, and TTL-based recovery.

For more demanding workflows, teams may use an outbox pattern, versioning, event-driven invalidation, or another carefully designed reconciliation mechanism.

Remember: a cache is an optimization. Its failure should not silently corrupt the authoritative business record.

---

# Layer 2 — Practical Engineering & Workflows

## 12. Inspect Persistence and Memory Configuration

Start by examining the running server.

```
INFO persistence
INFO memory
CONFIG GET maxmemory
CONFIG GET maxmemory-policy
```

Configuration commands may be restricted by server policy or managed-service permissions.

Inspect the results before making changes. Do not blindly apply production settings to a local or managed instance.

### Inspect key TTLs

```
TTL product:1042:details
PTTL product:1042:details
```

**PTTL** reports remaining lifetime in milliseconds.

### Inspect the keyspace

```
INFO keyspace
```

This provides a high-level view of database key counts and expiration statistics. It does not replace workload-specific monitoring.

## 13. Configure a Local Redis Experiment

For a disposable local instance, you can inspect or configure memory behavior for a controlled experiment.

First inspect the current settings:

```
CONFIG GET maxmemory
CONFIG GET maxmemory-policy
```

If your local server permits runtime configuration changes, you can set a small memory limit and an eviction policy for experimentation. Choose values that are appropriate for your environment and reset them after the lab if necessary.

For example, on a disposable instance:

```
CONFIG SET maxmemory 50mb
CONFIG SET maxmemory-policy allkeys-lru
```

This is a lab example, not a recommended universal production configuration. A small memory limit may cause immediate eviction or rejected writes depending on the dataset and policy.

Observe:

```
INFO memory
INFO stats
```

Then populate test keys and examine how the dataset behaves as memory pressure increases.

**Safety:** don't experiment with memory limits or eviction settings on a shared or production Redis instance without an approved change procedure.

## 14. Build a Reliable Cache-Aside Write Workflow

A simple product-update workflow should define what happens if cache invalidation fails.

Pseudocode:

```
@Transactional
public Product updateProduct(long id, ProductUpdate update) {
    Product product = productRepository.findById(id)
        .orElseThrow(() -> new ProductNotFoundException(id));

    product.apply(update);

    return productRepository.save(product);
}
```

After the database transaction commits, an application service can attempt to invalidate the corresponding cache key.

Conceptually:

```
Database transaction commits
           |
           v
Invalidate product:<id>
           |
           +---- Success --> Normal operation
           |
           +---- Failure --> Record failure and recover
```

A production design must consider transaction boundaries. Calling Redis inside a database transaction does not make the two operations part of one atomic transaction.

Possible recovery mechanisms include:

- Logging and alerting on invalidation failures.
- Retrying failed invalidations.
- A durable outbox event for important invalidation workflows.
- Short TTLs as a secondary bound on stale data.
- Reconciliation or versioning where requirements justify it.

Do not swallow every Redis exception silently. Decide whether the endpoint can safely succeed when the cache update fails and make that decision observable.

## 15. TTL Strategy in Application Code

Use a TTL that reflects the freshness requirements and expected workload.

Example:

```
Duration productDetailsTtl = Duration.ofMinutes(5);
```

When many keys are populated at the same time, identical TTLs can cause synchronized expiration patterns.

A common mitigation is to add a small randomized variation to expiration times, often called TTL jitter.

Conceptually:

```
Base TTL: 300 seconds
Random variation: 0–30 seconds

Actual TTL: 300–330 seconds
```

The exact range should be selected for the workload. Jitter reduces synchronized expiration risk; it does not prevent all cache stampedes.

### Avoid these TTL mistakes

- Creating cache entries with no expiry unintentionally.
- Applying one TTL to every kind of data.
- Using TTL as the only correctness mechanism for critical records.
- Assuming that an expired key will never be read by an already-running request.
- Forgetting to refresh or invalidate cached data after a write.
- Setting extremely short TTLs that make the cache ineffective.

## 16. Production Debugging Commands

When Redis behavior looks wrong, collect evidence before changing configuration.

```
PING
INFO memory
INFO stats
INFO persistence
INFO clients
INFO keyspace
SLOWLOG GET 10
LATENCY LATEST
```

Availability of diagnostic commands and their output depend on the Redis version and deployment permissions.

Useful questions:

1. Is Redis reachable?
2. Is memory usage close to the configured limit?
3. Are evictions increasing?
4. Are rejected writes increasing?
5. Are persistence operations causing resource pressure?
6. Are slow commands appearing?
7. Are client connections or buffers growing unexpectedly?
8. Is the cache hit ratio falling?
9. Is PostgreSQL receiving more requests than normal?
10. Did a recent deployment change cache keys, TTLs, or serialization?

### Example incident: sudden increase in database load

Symptoms:

- PostgreSQL CPU rises.
- API latency increases.
- Redis remains reachable.
- Cache hit ratio falls sharply.

Investigation:

1. Check whether a deployment changed key names.
2. Confirm that the expected keys are being populated.
3. Inspect TTLs and eviction statistics.
4. Examine whether a popular key expired or stopped being cached.
5. Compare application and database request metrics.
6. Fix the root cause and verify the hit ratio and database load recover.

Do not immediately increase Redis memory. A broken cache-key convention can cause a miss storm even when memory is plentiful.

## 17. Hands-On Lab: Eviction and Expiration

Use a disposable Redis instance.

### Task A — Compare TTL and eviction

1. Create a key with an expiry.

2. Create another key without an expiry.
3. Inspect both using `TTL`.
4. Configure a small memory limit and an appropriate policy in the disposable environment.
5. Generate enough test data to create memory pressure.
6. Observe which keys remain.
7. Restore the normal configuration after the experiment.

### Task B — Persistence investigation

Inspect:

```
INFO persistence
```

Identify:

- Whether RDB or AOF is enabled.
- When the most recent successful snapshot occurred, where applicable.
- Whether background persistence is active.
- Whether persistence errors are reported.

Do not infer that data is fully protected just because persistence is enabled.

### Task C — Cache invalidation design

Write a short design for this sequence:

```
Client updates product price
PostgreSQL commits the update
Redis invalidation fails
Another client requests the product
```

Your design should state:

- Which system is authoritative.
- Whether the update endpoint succeeds.
- How the invalidation failure is recorded.
- How stale data is bounded or prevented.
- How the system recovers.

### Task D — Capacity review

For a cache with a configured memory limit, document:

- Expected number of keys.
- Approximate value sizes.
- Key and object overhead assumptions.
- Expected TTL distribution.
- Eviction policy.
- Peak workload.
- Alert thresholds and response steps.

The objective is not to calculate a perfect number from assumptions. It is to build a capacity model that can be validated using measurements.

---

# Layer 3 — Production Engineering & Interview Mastery

## 18. Production Scenario: Redis Restarts and the Cache Is Empty

### Symptoms

- Redis restarts.
- Cache hit ratio drops.
- PostgreSQL traffic spikes.
- API latency increases.

### Likely explanation

The application relied on cached values that were lost or not restored after the restart.

For an ordinary cache, this can be an expected recovery scenario. The real issue may be that the application was not prepared for the sudden increase in database traffic.

### Response

1. Confirm Redis health and restart status.
2. Inspect persistence and recovery logs if the workload requires persistence.
3. Measure cache misses and PostgreSQL load.
4. Protect PostgreSQL with appropriate concurrency limits or load shedding if needed.
5. Allow the cache to warm gradually or repopulate it safely.
6. Verify that the system recovers without creating another overload.

### Prevention

- Load-test cold-cache behavior.
- Set realistic database capacity assumptions.
- Design gradual cache warming where useful.
- Monitor hit ratio, database load, and API latency together.
- Decide whether persistence is required for the actual Redis workload.

## 19. Production Scenario: Writes Start Failing

### Symptoms

- Redis write commands return errors.

- The application logs indicate memory-related failures.
- Reads may continue to work.

### Investigation

1. Inspect `INFO memory`.
2. Check `maxmemory` and `maxmemory-policy`.
3. Inspect eviction and rejected-command statistics.
4. Check whether the dataset has grown unexpectedly.
5. Identify keys that are unexpectedly large or lack expiry.
6. Verify that the chosen policy is appropriate for the workload.

### Corrective options

Depending on the findings:

- Remove unintended unbounded data.
- Add suitable TTLs to disposable entries.
- Reduce value sizes or duplicate caching.
- Adjust the memory budget after reviewing host capacity.
- Change the eviction policy if the current policy does not match the intended workload.
- Split unrelated workloads where their memory and reliability requirements conflict.

Do not automatically delete arbitrary keys to restore service. First determine whether the data is disposable cache state or important application state.

## 20. Production Scenario: Stale Product Prices

### Symptoms

Customers report old prices even after a successful product update.

### Investigation

1. Verify the authoritative value in PostgreSQL.
2. Read the corresponding Redis key.
3. Check the TTL and key naming.
4. Inspect all write paths that can modify the product.
5. Look for failed invalidations or a race between cache population and deletion.
6. Check whether multiple application versions use different key formats.

### Remediation

Choose an approach based on the required consistency:

- Correct the invalidation logic.
- Retry failed invalidations reliably.
- Introduce versioning where appropriate.

- Reduce TTL if the acceptable stale window is too large.
- Bypass the cache for operations that require authoritative reads.

The fix must address the underlying inconsistency rather than merely deleting the currently stale key.

## 21. Production Scenario: Redis Looks Healthy, but the API Is Slow

Redis may be healthy at the server level while the application remains slow.

Possible causes:

- High cache-miss rates.
- Slow serialization and deserialization.
- Large cached payloads.
- Network latency between the application and Redis.
- Excessive round trips caused by fetching fields one at a time.
- Slow database queries on cache misses.
- Connection-pool saturation.
- A hot key or contention around application-level coordination.

Investigate end-to-end latency rather than relying on a single Redis health check.

Measure the complete request path:

```
API request
   |
   +--> Redis lookup latency
   |
   +--> Database fallback latency
   |
   +--> Serialization / application processing
   |
   +--> Response latency
```

Optimize the actual bottleneck. A cache that saves 2 ms but introduces a complex failure mode may not be worth using for every endpoint.

## 22. Production Readiness Checklist

Before deploying Redis caching:

- [ ] The source of truth is explicitly documented.
- [ ] Every cached resource has a key-naming convention.
- [ ] TTLs reflect data freshness requirements.
- [ ] Cache invalidation is defined for every relevant write path.
- [ ] Missing cache entries are handled correctly.
- [ ] Serialization is consistent and safe.
- [ ] Memory limits and eviction policy match the workload.
- [ ] Redis outages have a documented degradation strategy.
- [ ] Database capacity has been evaluated for cold-cache conditions.
- [ ] Hit ratio, latency, memory, evictions, and errors are monitored.

- [ ] Credentials and network access are protected.
- [ ] Persistence and backups are configured if the workload requires them.
- [ ] Recovery procedures have been tested.
- [ ] Load tests include cold-cache and cache-degradation scenarios.
- [ ] Failure paths are covered by automated tests.

## 23. Advanced SDE-1 Interview Questions

### Q1. What is the difference between RDB and AOF?

RDB stores point-in-time snapshots. AOF records write operations for recovery through replay. Their recovery-point, recovery-time, and resource trade-offs differ.

### Q2. Does Redis persistence guarantee zero data loss?

No. The outcome depends on persistence configuration, when data was successfully written, the storage system, and the failure scenario.

### Q3. What is a Redis eviction policy?

It determines how Redis selects keys for removal when memory pressure requires eviction and the configured policy allows it.

### Q4. What is the difference between LRU and LFU?

LRU focuses on recency of access. LFU focuses on frequency of access. Both are approximations and perform differently under different workloads.

### Q5. Can a key be evicted before its TTL expires?

Yes. If memory pressure and the configured eviction policy permit it, a key can be removed before its expiration time.

### Q6. What happens when `noeviction` is configured and memory is exhausted?

Redis does not evict keys under that policy. Commands requiring additional memory may fail.

### Q7. Why is cache invalidation difficult?

Because the database and cache are separate systems, and concurrent reads, writes, and failures can leave them inconsistent.

### Q8. What is the dual-write problem?

An application updates two separate systems, and one operation may succeed while the other fails. Without coordination or recovery, their states can diverge.

### Q9. What is write-behind caching?

The cache accepts updates and the backing store is updated asynchronously later. It can improve write latency but introduces durability and recovery risks.

### Q10. How would you prevent a cache stampede?

Consider request coalescing, TTL jitter, background refresh, or carefully designed coordination. Select based on workload and failure behavior.

### Q11. Is a replica a backup?

Not necessarily. Replication improves availability and may aid recovery, but destructive or incorrect changes can propagate. Independent backups and tested restoration are still important.

### Q12. Why might a cache hit ratio fall suddenly?

Potential causes include mass expiration, eviction, a deployment changing key names, cache population failures, a workload shift, or a cold-cache restart.

### Q13. Why should cache memory be budgeted below host memory?

Redis needs overhead for connections, buffers, persistence, replication, and other processes. Operating at the host's full memory limit risks resource exhaustion.

### Q14. When is a cache inappropriate?

When the overhead, invalidation complexity, stale-data risk, or operational burden outweighs the benefit, or when the operation requires authoritative data and cannot tolerate the cache's consistency model.

### Q15. How would you prove that Redis improved performance?

Compare equivalent workloads before and after deployment, measuring API latency percentiles, database load, cache hit ratio, Redis latency, error rates, and correctness.

---

## 24. Mastery Assessment

Answer these in your own words:

1. Why can Redis lose recent writes despite persistence being enabled?
2. Why is `volatile-lru` potentially unsuitable when most keys have no TTL?
3. How could a cache invalidation failure create stale data?
4. What is the difference between an eviction and an expiration?
5. Why does a cache restart sometimes overload PostgreSQL?
6. When would you choose cache-aside over write-behind?
7. What would you investigate if Redis memory is stable but cache hit ratio collapses?
8. What is the difference between replication and a backup?

**9.** How can TTL jitter help, and what does it fail to guarantee?

**10.** Why is updating PostgreSQL and Redis not automatically an atomic operation?

## Completion Checklist

- [ ] I can explain RDB and AOF.
- [ ] I can distinguish persistence, replication, and backups.
- [ ] I can inspect memory and persistence statistics.
- [ ] I understand `maxmemory` and major eviction policies.
- [ ] I can distinguish expiration from eviction.
- [ ] I can explain cache invalidation and the dual-write problem.
- [ ] I can compare cache-aside, read-through, write-through, and write-behind.
- [ ] I can diagnose a cold-cache database overload.
- [ ] I can reason about memory pressure and rejected writes.
- [ ] I can design a measured, recoverable caching strategy.

---

## What Comes Next?

**Redis — Chunk 3 of 3: Production Reliability, Security, Performance & Incident Mastery**

The final Redis chapter will bring the pieces together: operational design, connection handling, hot keys, pipelining, transactions and atomicity, rate limiting, distributed coordination, observability, security, incident simulations, and final interview mastery.

**Repository workflow:** Save this chapter as `content/redis/02-redis-persistence-eviction-caching-patterns.md`. Do not generate a PDF yet. Generate `docs/redis/BEH_REDIS_COMPLETE_HANDBOOK.pdf` only after all three Redis chapters have been saved and verified.

**PART III: PRODUCTION RELIABILITY, SECURITY, PERFORMANCE & INCIDENT MASTERY**

Chunk 3 of 3 · Backend Engineering Handbook

# Redis — Chunk 3: Production Reliability, Security, Performance & Incident Mastery

## Backend Engineering Tools & Infrastructure Handbook

**Phase:** II — Backend Infrastructure

**Chunk:** 3 of 3

**Source file:** `content/redis/03-redis-production-reliability-security-performance.md`

---

## Learning Objectives

By the end of this chapter, you should be able to:

- Design a reliable Redis integration for a Spring Boot backend.
- Diagnose hot keys, slow commands, connection exhaustion, and latency spikes.
- Explain pipelining, transactions, atomic commands, and Lua scripting.
- Implement a basic Redis-backed rate limiter.
- Understand Redis replication, Sentinel, and Cluster at a conceptual level.
- Apply practical Redis security and observability controls.
- Respond systematically to Redis-related production incidents.
- Defend caching, rate limiting, and coordination trade-offs in SDE-1 interviews.

---

# Layer 1 — Fundamentals & Mental Models

## 1. Redis in a Production Architecture

In production, Redis is not just a fast key-value store. It is an infrastructure dependency that introduces its own failure modes, capacity constraints, and operational responsibilities.

Consider a Spring Boot service backed by PostgreSQL:

```
                  Client
                    |
                    v
                Nginx / LB
                    |
                    v
             Spring Boot API
               /        \
              v          v
           Redis     PostgreSQL
        cache and    source of
        counters       truth
```

Redis might support:

- Caching frequently requested product data.
- Rate limiting requests.
- Storing short-lived verification state.
- Tracking counters and selected ephemeral state.
- Coordinating certain distributed workflows.

PostgreSQL remains the authoritative source for business records in this example.

The application must still define what happens when Redis is slow, unavailable, full, or returning stale data.

### The three questions for every Redis integration

Before adding Redis to an endpoint, answer:

1. **Correctness:** What happens if the value is missing, stale, or evicted?
2. **Reliability:** What happens if Redis cannot be reached?
3. **Performance:** Does Redis reduce total request cost, or introduce another bottleneck?

If these questions have no clear answers, the integration is not production-ready.

## 2. Atomicity: Why Separate Commands Can Be Dangerous

Suppose an application implements a counter using two operations:

```
GET page:home:views
```

Then the application increments the result and writes it back:

```
SET page:home:views 101
```

Two requests can read the same old value and overwrite each other's updates.

```
Initial value = 100

Request A reads 100
Request B reads 100

Request A writes 101
Request B writes 101

Expected: 102
Actual:   101
```

This is a lost update.

Use Redis's atomic increment operation instead:

```
INCR page:home:views
```

Redis executes each individual command atomically with respect to other commands on the server. That does not mean an arbitrary sequence of commands is automatically atomic.

### Key distinction

- **`INCR`**: one atomic increment operation.
- **`GET`** followed by **`SET`**: two separate operations with a race window.
- A Lua script: can execute a group of Redis operations atomically on the server.
- **`MULTI`**/**`EXEC`**: queues and executes a transaction's commands together, but does not provide rollback semantics like a relational database transaction.

Use the smallest mechanism that satisfies the correctness requirement.

## 3. Pipelining vs. Transactions vs. Lua

These three mechanisms solve different problems.

### Pipelining

Sends multiple commands over the network without waiting for each response before sending the next. This significantly reduces network round trips and socket I/O overhead.

- **Use when:** Commands can be issued independently and you want to reduce batch network latency.
- **Limitation:** Does not guarantee atomicity; other clients' commands can interleave between pipelined commands on the server.

### Transactions — `MULTI` / `EXEC`

Queues multiple commands on the server and executes them sequentially as a single block without other clients' commands interleaving between the queued commands during execution.

- **Use when:** Multiple commands must execute sequentially without interruption.
- **Limitation:** Does not provide relational-style rollback for runtime command errors (commands after a runtime failure will still execute).

### Lua Scripting

Runs procedural application logic directly on the Redis server as a single atomic operation relative to other Redis commands.

- **Use when:** Complex conditional logic, branching, or read-then-write decisions must happen atomically (e.g., checking and incrementing a rate-limit counter).
- **Limitation:** Long-running Lua scripts block the entire Redis server event loop while running.

### Example: pipelining

Suppose an application needs to read 100 independent keys.

Without pipelining, it may perform 100 request-response round trips. With pipelining, it can send commands in batches and collect their responses.

Pipelining reduces network overhead but does not make the entire batch a transaction. Other clients may execute commands between the pipelined commands.

### Example: transaction

```
MULTI
INCR order:9001:attempts
EXPIRE order:9001:attempts 3600
EXEC
```

The queued commands execute together when `EXEC` runs. This is useful when you need to group operations, but understand the limitations:

- Commands are queued before execution.
- Runtime errors do not automatically roll back all successful commands.
- A transaction does not create a general-purpose atomic transaction spanning Redis and PostgreSQL.

For more complicated conditional operations, Lua or a suitable optimistic-locking design may be more appropriate.

## 4. Hot Keys and Hot Partitions

A hot key is a key receiving disproportionately high traffic.

Example:

```
product:popular-item
```

Suppose a product becomes viral and millions of requests access the same key.

Even when Redis handles the command efficiently, the concentrated request volume can cause bottlenecks in the application, network, or Redis deployment.

In a Redis Cluster, a hot key is especially important because a key maps to a single hash slot, which is served by one cluster shard at a time.

### Common mitigations

- Cache at an upstream layer where appropriate.

- Use local in-process caching for carefully selected, tolerably stale data.
- Reduce redundant reads through request coalescing.
- Avoid fetching oversized values.
- Consider distributing read load when the data model and consistency requirements permit it.
- Investigate whether a single key is being used as a global counter or coordination bottleneck.

Do not split a key into multiple independent counters without designing how to aggregate them correctly. Distribution may improve throughput but complicate reads and updates.

## 5. Connection Pools and Latency

A Redis request usually involves more than executing a command.

The application may need to:

1. Acquire a connection.
2. Send the command over the network.
3. Wait for Redis to execute it.
4. Read the response.
5. Deserialize the result.
6. Release the connection.

A slow API can therefore be caused by connection acquisition, network latency, Redis processing, or application-side work.

### Common warning signs

- Connection-pool wait time rises.
- Redis command latency remains low, but API latency is high.
- Many requests time out simultaneously.
- Client connections increase sharply.
- Large responses cause excessive processing.
- Slow database fallbacks follow cache misses.

Increasing the connection pool without investigating can make things worse. More concurrent requests may increase resource pressure on Redis and the application.

Tune pool sizes, timeouts, and concurrency using measured workload behavior.

## 6. Redis Replication, Sentinel, and Cluster

These concepts address different operational needs.

| Capability | Main purpose | Important limitation |
| --- | --- | --- |
| Replication | Copies data from a primary to replicas | Replication is generally asynchronous and does not guarantee zero data loss |
| Sentinel | Monitors a non-clustered Redis primary/replica setup and supports automatic failover | It does not shard the dataset |

| Capability | Main purpose | Important limitation |
| --- | --- | --- |
| Redis Cluster | Partitions the keyspace across shards and provides replication and failover mechanisms | Adds operational and application-design complexity |

### Replication

A primary processes writes and replicates changes to replicas. Replicas can support selected read workloads, but replication lag can make replica reads stale.

### Sentinel

Sentinel monitors Redis instances, helps detect primary failures, and coordinates failover. Clients must support the appropriate discovery and reconnection behavior.

### Cluster

Redis Cluster partitions keys into hash slots distributed across multiple primary nodes. Replicas can provide redundancy and failover.

Cluster introduces considerations such as:

- Key placement.
- Multi-key operations and hash slots.
- Client redirection.
- Resharding.
- Hot shards.
- Partial failures.

For a small backend project, a single Redis instance may be sufficient. For a production service with explicit availability or capacity requirements, choose the topology based on measured needs rather than assuming a cluster is always better.

---

# Layer 2 — Practical Engineering & Workflows

## 7. Implement a Basic Redis Rate Limiter

Rate limiting controls how frequently a caller can perform an operation.

Example policy:

> Allow at most 100 requests per user per minute.

A simple fixed-window design stores a counter for each user and time window.

Key:

```
rate:user:87:202610230930
```

The timestamp here is illustrative; the actual key should be derived from the application's chosen window boundaries.

## Basic command sequence

A naive implementation might do this:

```
INCR rate:user:87:window
EXPIRE rate:user:87:window 60
```

The counter increments atomically, but these two separate commands create a failure window. If the increment succeeds and the process fails before setting the expiry, the key could remain without a TTL.

A stronger implementation should ensure that the counter and its expiry are managed together.

## Lua implementation

The following script increments a counter and establishes an expiry on the first increment:

```
local current = redis.call("INCR", KEYS[1])

if current == 1 then
    redis.call("EXPIRE", KEYS[1], ARGV[1])
end

return current
```

Call it with:

- `KEYS[1]`: the rate-limit counter key.
- `ARGV[1]`: the window duration in seconds.

The script executes atomically relative to other Redis commands. If the first increment occurs, it sets the expiry before returning.

## Application decision

The application compares the returned count against the configured limit.

```
count <= 100  -> Allow
count > 100   -> Reject or throttle
```

A typical API may return HTTP `429 Too Many Requests` when the limit is exceeded.

## Fixed-window trade-off

A fixed window can permit a burst near a boundary. A client might consume most of its allowance just before a window ends and then immediately consume the next window's allowance.

For stricter rate limiting, consider sliding-window or token-bucket approaches. Each has different complexity, memory, and burst-control characteristics.

**Security note:** do not trust a caller-supplied user ID as the rate-limit identity without authenticating and validating it. For unauthenticated endpoints, consider appropriate IP- or token-based controls, accounting for proxies and shared addresses.

## 8. A Spring Boot Rate-Limiter Workflow

A clean design separates Redis operations from HTTP policy.

```
Incoming HTTP request
        |
        v
Authentication / identity resolution
        |
        v
RateLimitService
        |
        v
Atomic Redis counter
        |
        v
Compare with configured limit
        |
        +---- Within limit --> Continue
        |
        +---- Over limit ----> HTTP 429
```

A simplified service contract might look like this:

```
public interface RateLimitService {
    long recordRequest(String subject, Duration window);
}
```

The implementation would call the Lua script using Spring Data Redis and return the count. A filter or interceptor would enforce the configured limit.

Design decisions to document:

- What identifies a caller?
- Is the limit per user, API key, tenant, or IP address?
- What is the time window?
- What happens if Redis is unavailable?
- How are limits applied consistently across multiple API instances?
- Are retry headers or reset information returned to clients?

A Redis-backed counter can coordinate rate limiting across application instances, unlike an ordinary in-memory counter in each process.

However, rate limiting is a security and availability control. If Redis fails, allowing all traffic may create an overload risk; rejecting all traffic may cause an outage. The appropriate fallback depends on the endpoint and threat model.

## 9. Diagnose Redis with CLI Commands

Start with basic health:

```
redis-cli PING
```

Inspect server information:

```
redis-cli INFO server
redis-cli INFO memory
redis-cli INFO stats
redis-cli INFO clients
redis-cli INFO persistence
```

Inspect slow commands:

```
redis-cli SLOWLOG GET 10
```

Inspect latency events:

```
redis-cli LATENCY LATEST
```

The latency-monitoring command depends on the server's configuration and permissions.

### Important diagnostic distinction

A command recorded in the Redis slow log indicates that server-side execution crossed the configured slow-log threshold. It does not capture the full end-to-end time spent waiting for a connection or traversing the network.

Therefore, compare:

- Application-side Redis latency.
- Redis server-side command latency.
- Connection-pool wait time.
- Network latency.
- API endpoint latency.

This comparison helps isolate where time is being spent.

## 10. Monitor Key-Level Memory and Keyspace Behavior

For a known key:

```
MEMORY USAGE product:1042:details
TTL product:1042:details
TYPE product:1042:details
```

These commands help answer:

- What data structure is stored?
- How much memory does this key use?
- Does it have an expiry?

Use incremental key iteration for operational exploration:

```
SCAN 0 MATCH product:* COUNT 100
```

`COUNT` is a work hint, not a guarantee that exactly 100 keys will be returned. Continue from the returned cursor until iteration completes.

Do not use broad key scans as a routine per-request application operation.

### Example: unexpected key growth

Suppose Redis memory rises continuously.

A useful investigation sequence:

1. Inspect memory and keyspace statistics.
2. Check whether expected TTLs are present.
3. Search for a known key family using `SCAN`.
4. Inspect representative keys with `MEMORY USAGE`.
5. Identify which application feature creates them.
6. Check whether cleanup or expiry behavior changed.
7. Correct the creation or lifecycle bug.
8. Confirm memory growth stabilizes.

Avoid deleting large groups of keys before understanding their purpose.

## 11. Redis Security: A Practical Checklist

Redis is commonly deployed as an internal service. It should not be exposed indiscriminately to the public internet.

### Network controls

- Restrict access to authorized application networks.
- Bind and firewall the service appropriately.
- Use private connectivity for managed Redis deployments.
- Use TLS when required by the deployment's security architecture.

### Authentication and authorization

- Configure authentication and ACLs where supported.
- Give each application only the permissions it needs.
- Separate application access from administrative access.
- Rotate credentials and keep them out of source control.

### Application controls

- Avoid logging credentials or sensitive cached values.
- Validate and constrain cache-key inputs.
- Use safe serialization formats and configurations.
- Set appropriate TTLs for sensitive transient data.
- Consider data residency and retention requirements.
- Avoid treating possession of a cache key as authorization to access a business resource.

A private network alone is not a complete security boundary.

## 12. Spring Boot Reliability Settings

A production Redis integration should define connection behavior explicitly.

Example configuration shape:

```
spring:
  data:
    redis:
      host: ${REDIS_HOST:localhost}
      port: ${REDIS_PORT:6379}
      connect-timeout: ${REDIS_CONNECT_TIMEOUT:2s}
      timeout: ${REDIS_COMMAND_TIMEOUT:2s}
```

Verify the exact property names and supported values against the Spring Boot version in your project. The values shown are illustrative starting points, not universal production recommendations.

Consider the following when tuning:

- Connection timeout.
- Command timeout.
- Connection-pool size and acquisition behavior.
- Retry policy.
- Reconnection behavior.
- Serialization overhead.
- Application concurrency.
- The consequence of falling back to PostgreSQL.

Retries need particular care. A timed-out command may already have executed on Redis even though the client did not receive its response. Blindly retrying a non-idempotent operation can apply it twice.

For example, retrying `INCR` after an ambiguous timeout can increment the counter twice. Idempotency or operation-specific retry logic may be required.

## 13. Reliability Patterns for Redis Clients

### Bounded timeouts

A request should not wait indefinitely for Redis. Timeouts should fit within the overall API latency budget.

### Controlled retries

Use bounded retries and backoff where appropriate. Do not create retry storms during a Redis outage.

### Bulkheads and concurrency limits

Limit how many requests can concurrently depend on a failing Redis service. This can prevent resource exhaustion from spreading through the application.

### Circuit breakers

A circuit breaker can temporarily stop repeated attempts to call an unhealthy dependency. It must be configured carefully: bypassing Redis can increase database traffic, and rate limiting may have different fallback requirements from ordinary caching.

### Observability

Record failures, latency, timeout counts, and fallback activity. A degraded cache should not remain invisible until PostgreSQL becomes overloaded.

---

# Layer 3 — Production Engineering & Interview Mastery

## 14. Incident Simulation: API Latency Suddenly Increases

### Symptoms

- API p95 latency rises from 80 ms to 900 ms.
- Redis is reachable.
- PostgreSQL CPU increases.
- Cache hit ratio drops.

### Investigation

**Step 1 — Verify Redis health**

```
redis-cli PING
redis-cli INFO stats
redis-cli INFO memory
```

**Step 2 — Check the application's Redis metrics**

Inspect:

- Command latency.
- Connection-pool acquisition time.
- Redis timeouts.
- Cache hits and misses.

**Step 3 — Check for key changes**

A deployment may have changed the cache-key prefix or serialization format, causing misses for every request.

**Step 4 — Inspect expiry and eviction behavior**

Check whether keys expire too quickly or are being evicted under memory pressure.

**Step 5 — Protect the database**

If fallback traffic is overwhelming PostgreSQL, apply the service's approved concurrency controls, load shedding, or temporary traffic restrictions.

### Resolution criteria

Do not consider the incident resolved merely because Redis responds to `PING`.

Verify:

- Cache hit ratio recovers.
- PostgreSQL load returns toward baseline.
- API p95 latency improves.
- Error rates stabilize.
- No correctness regression was introduced by the mitigation.

## 15. Incident Simulation: Redis Is Unavailable

### Symptoms

- Connection timeouts rise.
- API requests begin failing or slowing down.
- PostgreSQL traffic may increase.

### Response

1. Determine whether the failure is limited to one application instance or affects the Redis service.
2. Check service health, network connectivity, connection limits, and recent changes.
3. Determine whether the application is falling back to PostgreSQL.
4. Protect the database if fallback traffic is excessive.
5. Restore connectivity or follow the managed service's failover procedure.
6. Verify client reconnection behavior.
7. Confirm cache recovery and normal traffic distribution.

### Design review

Before the incident, the team should have decided:

- Which endpoints may bypass the cache.
- Which operations require Redis to enforce a security or traffic policy.
- How fallback load is controlled.
- Whether the cache is rebuildable.
- How to detect a Redis outage quickly.

A cache outage and a rate-limiter outage are not necessarily equivalent. Failing open may be reasonable for a noncritical cache but unsafe for some abuse-sensitive endpoints.

## 16. Incident Simulation: A Hot Key Overloads Redis

### Symptoms

- One key receives extreme request volume.
- Application latency increases.
- Redis command processing or network utilization rises.
- Most unrelated keys may still behave normally.

### Investigation

1. Identify the popular resource or operation from application metrics.
2. Determine whether requests are concentrated on one Redis key.
3. Inspect value size and command type.
4. Check whether each request unnecessarily retrieves a large payload.
5. Determine whether upstream or local caching is safe.
6. Evaluate whether read traffic can be distributed without violating consistency requirements.

### Possible mitigations

- Reduce duplicate reads.
- Cache at a suitable upstream layer.
- Use local caching with an explicit staleness policy.
- Split independent data where the model permits it.
- Avoid using one key as a global coordination point.

The mitigation should preserve the semantics of the data. A global counter, inventory value, and public product description have very different requirements.

## 17. Incident Simulation: Rate Limiting Stops Working

### Symptoms

- Clients exceed their expected limits.
- Rate-limit counters disappear unexpectedly.
- Different application instances enforce inconsistent limits.

### Investigation

1. Verify the key format and caller identity.
2. Check counter TTLs.
3. Confirm that all application instances use the same Redis deployment and policy.
4. Check script or command failures.
5. Review retry behavior and timeout handling.
6. Determine whether counters are being evicted or reset during failover.
7. Confirm that the API enforcement code interprets the counter correctly.

### Prevention

- Test concurrent requests.
- Ensure counter creation and expiry are handled atomically.
- Monitor script failures and rate-limit decisions.
- Define behavior during Redis failover.
- Ensure user-controlled identifiers cannot bypass or poison rate-limit keys.

A rate limiter is only as correct as its identity model, counter logic, expiry policy, and failure behavior.

## 18. Redis Performance Optimization Checklist

Before making changes, establish a baseline.

### Application layer

- [ ] Measure cache hit and miss ratios.
- [ ] Measure connection acquisition and command latency.
- [ ] Avoid unnecessary round trips.
- [ ] Avoid fetching large payloads when only a small field is required.
- [ ] Check serialization and deserialization cost.
- [ ] Use pipelining when commands are independent and batching is beneficial.
- [ ] Apply concurrency limits when needed.

### Redis layer

- [ ] Inspect memory usage and eviction statistics.
- [ ] Check slow-log entries and latency events.
- [ ] Investigate hot keys and oversized values.
- [ ] Review persistence overhead if persistence is enabled.
- [ ] Verify that commands are appropriate for the data structure.
- [ ] Avoid expensive full-keyspace operations in latency-sensitive paths.

### System layer

- [ ] Measure network latency between application and Redis.
- [ ] Verify that the deployment has adequate CPU and memory.
- [ ] Inspect connection counts and resource limits.
- [ ] Review failover and reconnect behavior.
- [ ] Load-test both warm-cache and cold-cache scenarios.

Optimize the bottleneck demonstrated by measurements, not the component that seems most interesting.

## 19. Redis and Kafka Are Not Interchangeable

Both systems can participate in distributed backend architectures, but their primary roles differ.

| Redis | Kafka |
|---|---|
| In-memory data structures and low-latency access | Durable event streaming and event-log workflows |
| Frequently used for caching and counters | Frequently used for event distribution and replay |
| Supports keys, TTLs, sets, hashes, sorted sets, and streams | Organizes records into topics and partitions |
| Cache eviction can remove entries | Retention policies govern stored event records |
| Redis Streams support consumer groups | Kafka provides consumer groups and partition-based processing |

Redis Streams can support event-processing workloads, and Kafka can be used in low-latency pipelines. The decision depends on retention, replay, throughput, durability, operational requirements, and the surrounding architecture.

Do not select Redis merely because the system needs asynchronous processing. Choose the technology that matches the required delivery and recovery model.

## 20. Production Readiness Checklist

Before considering a Redis integration production-ready:

- [ ] The Redis use case and authoritative data source are documented.
- [ ] Every key family has a naming and expiry policy.
- [ ] Memory limits and eviction behavior are understood.
- [ ] Connection and command timeouts are bounded.
- [ ] Retry behavior accounts for ambiguous command outcomes.
- [ ] Hot-key and cache-stampede risks have been considered.
- [ ] The application handles cache misses correctly.
- [ ] Redis outages have a tested degradation strategy.
- [ ] Database fallback capacity has been evaluated.
- [ ] Rate limiting has correct identity and concurrency semantics.
- [ ] Credentials, network access, and permissions are controlled.
- [ ] Relevant latency, memory, hit-rate, and error metrics are monitored.
- [ ] Persistence and recovery are configured if required by the workload.
- [ ] Failover and restart behavior have been tested.
- [ ] Runbooks describe the first steps for common incidents.

## 21. SDE-1 Interview Questions & Answers

### Q1. What is the difference between atomicity and pipelining?

Atomicity concerns whether operations execute as an indivisible unit relative to other commands. Pipelining reduces network round trips by sending multiple commands without waiting for every response. Pipelining alone does not make a batch atomic.

### Q2. Does Redis `MULTI`/`EXEC` provide rollback like PostgreSQL?

No. Redis transactions group queued commands for execution, but runtime command errors do not automatically roll back other successful commands.

### Q3. When should you use Lua scripting?

When multiple Redis operations must execute atomically, particularly when the operation requires a conditional decision based on Redis state.

### Q4. What is a hot key?

A key that receives disproportionately high traffic and becomes a potential bottleneck.

### Q5. Why can increasing the connection pool make performance worse?

It can increase concurrency and resource pressure on Redis and the application, worsening contention or overload instead of fixing the underlying bottleneck.

### Q6. What is the difference between Redis Sentinel and Redis Cluster?

Sentinel provides monitoring and failover coordination for a primary/replica deployment. Redis Cluster partitions the keyspace across shards and supports replication and failover mechanisms.

### Q7. Why can retrying a timed-out `INCR` be dangerous?

The original increment may have executed even though its response was lost. Retrying can increment the counter a second time.

### Q8. How would you implement a basic distributed rate limiter?

Use a shared Redis counter keyed by caller identity and time window. Increment it atomically, ensure its expiry is set safely, and compare the result against the configured limit.

### Q9. What is the main limitation of a fixed-window rate limiter?

It can allow bursts around window boundaries, since a caller may consume the allowance in adjacent windows close together.

### Q10. Why is `PING` insufficient for diagnosing a slow API?

It verifies basic responsiveness but does not measure application connection waits, cache hit rate, serialization, database fallback, or end-to-end request latency.

### Q11. How do you distinguish Redis server latency from client-side latency?

Compare Redis-side slow-log and latency information with application-side timings, connection-pool wait time, network measurements, and total endpoint latency.

### Q12. How would you respond to a Redis outage?

Confirm the scope, inspect service and network health, assess fallback load, protect downstream systems, restore or fail over Redis, and verify recovery through application metrics.

### Q13. Why might rate limiting need a different outage policy from caching?

Bypassing an ordinary cache may only increase database work. Bypassing a rate limiter can remove an abuse or overload control, so its failure policy must account for security and availability.

### Q14. What is a hot partition in Redis Cluster?

A shard may receive disproportionate traffic because its assigned keys or slots are unusually popular. This can create a bottleneck even if other shards are underutilized.

### Q15. How do you prove a Redis optimization worked?

Use a controlled workload and compare latency percentiles, request throughput, error rates, Redis resource usage, database load, and correctness before and after the change.

---

## 22. Final Mastery Assessment

Try these without looking at the answers above.

1. A Redis command times out after incrementing a counter. How would you avoid accidentally incrementing it twice?
2. Explain why pipelining improves throughput without guaranteeing atomicity.
3. A popular product key overwhelms one Redis Cluster shard. Propose two mitigations and explain their trade-offs.
4. Redis is healthy, but API latency is high. What metrics would help separate connection, network, server, and application bottlenecks?
5. Your distributed rate limiter must enforce 100 requests per minute. Why should counter increment and initial expiry be handled atomically?
6. What could happen if Redis fails and every application instance falls back to PostgreSQL?
7. Why might a circuit breaker help during an outage, and why can bypassing Redis still be dangerous?
8. What is the difference between Redis replication, Sentinel, and Cluster?
9. When would you prefer Redis caching over repeatedly querying PostgreSQL, and when might caching be the wrong choice?
10. Design an incident response for a sudden drop in cache hit ratio following a deployment.

## 23. Redis Handbook Completion

The Redis curriculum is now complete across three planned chunks:

| Chunk | Content | Source |
|---|---|---|
| 1 | Caching fundamentals, data structures, mental models | `content/redis/01-redis-caching-fundamentals-data-structures.md` |

| Chunk | Content | Source |
|---|---|---|
| 2 | Persistence, eviction, TTL strategy, caching patterns, consistency | `content/redis/02-redis-persistence-eviction-caching-patterns.md` |
| 3 | Production reliability, security, performance, incident mastery | `content/redis/03-redis-production-reliability-security-performance.md` |

### Finalization protocol

1. Save all three Markdown sources in the repository.
2. Verify that each file contains the complete chapter.
3. Review command examples, configuration, formatting, and cross-references.
4. Assemble the three chapters into one complete Redis handbook.
5. Generate and inspect `docs/redis/BEH_REDIS_COMPLETE_HANDBOOK.pdf`.
6. Update the README only after source and PDF verification.
7. Commit the verified Redis deliverables with the exact commit message:

```
Redis done
```

Do not create individual PDFs for the three chunks.

---

## Next Tool: Kafka — Chunk 1 of 3

The next handbook begins **Kafka Fundamentals, Event Streaming & Core Mental Models**. It will cover event-driven architecture, producers, consumers, topics, partitions, offsets, brokers, and how Kafka differs from Redis and ordinary message queues.